REST API Contract — Simplified Edition

University of Ruhuna · Software Architecture 2026 · v1.1 (University Project Build)

SIMPLIFICATIONS APPLIED TO THIS VERSION

1. **No Refresh Tokens** — POST /auth/refresh removed. Single JWT token expires in 24 h.
2. **Mock Payment Gateway** — POST /payments/mock-confirm replaces PayHere + webhook. Returns SUCCESS immediately.
3. **No District CRUD** — GET /districts kept for reference. POST/PUT removed. Frontend hardcodes the 25 districts.
4. **No Geospatial** — location.lat / location.lng fields dropped. location.address is an optional string only.

Conventions & Standards

Base URL	https://api.trafficfines.lk/api/v1
Request/Response	application/json
Auth Header	Authorization: Bearer <token>
Token Strategy	Single JWT access token — 24 h expiry. No refresh token.
Dates	ISO 8601 — e.g. 2026-05-13T10:30:00Z
IDs	MongoDB ObjectId strings (24 hex chars)
Pagination	?page=1&limit=20 → { data, total, page, pages }
Error Envelope	{ "success": false, "message": "...", "errors": [] }
Success Envelope	{ "success": true, "data": { ... } }

Code	Meaning
200	OK — Successful read / update
201	Created — Resource created
204	No Content — Delete success
400	Bad Request — Validation error
401	Unauthorized — Missing / invalid JWT
403	Forbidden — Insufficient role
404	Not Found
409	Conflict — Duplicate resource
422	Unprocessable — Business rule violation
500	Internal Server Error

Role	Description
DRIVER	Civilian paying traffic fines
OFFICER	Traffic police officer issuing fines
ADMIN	Senior official — analytics only
PUBLIC	No authentication required

1. Authentication /api/v1/auth

✂ **Removed** POST /auth/refresh — refresh token endpoint removed. Single 24-hour JWT is sufficient for a university project.

Method	Endpoint	Description	Auth
POST	`/auth/register`	Register a new user (Driver / Officer)	Public
POST	`/auth/login`	Login — receive 24-hour JWT access token	Public

POST	/auth/logout	Invalidate session (client discards token)	JWT
GET	/auth/me	Get currently authenticated user profile	JWT
PUT	/auth/change-password	Change own password	JWT
POST	/auth/forgot-password	Request password reset OTP via SMS	Public
POST	/auth/reset-password	Reset password using OTP	Public

Detailed Contracts

POST /auth/register — Auth: Public

Request Body:

```
{
"fullName": "Kamal Perera",
"email": "kamal@example.com",
"phone": "0771234567",
"password": "Str0ngPass1",
"role": "DRIVER", // DRIVER | OFFICER
"licenseNo": "B1234567", // Required if DRIVER
"badgeNo": "SLP-00123", // Required if OFFICER
"districtId": "64f1a2b3c4d5e6f7a8b9c0d1" // Required if OFFICER
}
```

Response Body:

```
{
"success": true,
"data": {
"userId": "64f1a2b3c4d5e6f7a8b9c0d1",
"fullName": "Kamal Perera",
"email": "kamal@example.com",
"role": "DRIVER",
"token": "eyJhbGci..." // 24-hour access token
}
}
```

■ Password must be min 8 chars with uppercase, lowercase and digit.

POST /auth/login — Auth: Public

Request Body:

```
{
"email": "kamal@example.com",
"password": "Str0ngPass1"
}
```

Response Body:

```
{
"success": true,
"data": {
"token": "eyJhbGci...", // JWT — expires in 24 h
"expiresIn": 86400, // seconds
"user": {
"userId": "64f...",
"fullName": "Kamal Perera",
```

```
"role": "DRIVER"
}
}
}
```

■ Single token returned — no refresh token. Client stores in memory or localStorage.
■ isActive: false accounts receive 403.

2. Fine Categories /api/v1/fine-categories

Method	Endpoint	Description	Auth
GET	/fine-categories	List all active fine categories	Public
GET	/fine-categories/:id	Get a single fine category by ID	Public
POST	/fine-categories	Create a new fine category	ADMIN
PUT	/fine-categories/:id	Update a fine category	ADMIN
DELETE	/fine-categories/:id	Soft-delete (deactivate) a category	ADMIN

GET /fine-categories — Auth: Public

Request Body:

```
Query: ?active=true|false (default: true)
```

Response Body:

```
{
"success": true,
"data": [
{
"categoryId": "64f1a2b3c4d5e6f7a8b9c0d2",
"code": "SP-01",
"name": "Speeding (1-20 km/h over limit)",
"amount": 1500.00,
"points": 2,
"isActive": true
}
]
}
```

3. Fines /api/v1/fines

Simplified location.lat and location.lng fields removed. location.address is an optional free-text string only.

Method	Endpoint	Description	Auth
POST	/fines	Officer issues a new traffic fine	OFFICER
GET	/fines/lookup	Public lookup by referenceNo + categoryCode	Public
GET	/fines/:id	Get full fine details by MongoDB ID	JWT
GET	/fines	List all fines (paginated, filterable)	ADMIN
GET	/fines/my	Driver's own fines list	DRIVER

GET	/fines/officer/issued	Fines issued by the logged-in officer	OFFICER
PATCH	/fines/:id/cancel	Cancel an unpaid fine	OFFICER / ADMIN

POST /fines — Auth: OFFICER

Request Body:

```
{
"vehicleNo": "CAB-1234",
"driverName": "Nimal Silva", // Optional
"driverLicense": "B7654321", // Optional
"categoryId": "64f1a2b3c4d5e6f7a8b9c0d2",
"districtId": "64f1a2b3c4d5e6f7a8b9c0d3",
"location": {
"address": "Galle Road, Colombo 3" // Optional free text
},
"notes": "Driver was overtaking on double line"
}
```

Response Body:

```
{
"success": true,
"data": {
"fineId": "64f1a2b3c4d5e6f7a8b9c0d4",
"referenceNo": "TF-20260513-000123",
"vehicleNo": "CAB-1234",
"categoryCode": "SP-01",
"amount": 1500.00,
"status": "PENDING",
"issuedAt": "2026-05-13T10:30:00Z",
"dueDate": "2026-06-12T23:59:59Z"
}
}
```

■ referenceNo and categoryCode are printed on the physical fine sheet handed to the driver.

GET /fines/lookup — Auth: Public

Request Body:

```
Query: ?referenceNo=TF-20260513-000123&categoryCode=SP-01
```

Response Body:

```
{
"success": true,
"data": {
"fineId": "64f1a2b3c4d5e6f7a8b9c0d4",
"referenceNo": "TF-20260513-000123",
"vehicleNo": "CAB-1234",
"category": { "code": "SP-01", "name": "Speeding (1-20 km/h over)", "amount": 1500.00 },
"status": "PENDING",
"issuedAt": "2026-05-13T10:30:00Z",
"dueDate": "2026-06-12T23:59:59Z",
"district": "Colombo",
"officer": { "badgeNo": "SLP-00123", "name": "Officer Bandara" }
```

```
}
}
```

■ Returns 404 if reference + category combination does not match.
■ Returns 422 if fine is already PAID or CANCELLED.

4. Payments /api/v1/payments — Mock Gateway

■ SIMPLIFIED — MOCK PAYMENT FLOW

Real PayHere integration and webhook handling have been replaced with a single mock endpoint. The mobile app / web portal sends payment details and the backend immediately responds with SUCCESS, marks the fine as PAID, generates a receipt, and triggers the SMS. No HMAC signature, no gateway redirect, no webhook callback.

Method	Endpoint	Description	Auth
POST	/payments/mock-confirm	Pay a fine (mock — always succeeds)	Public / DRIVER
GET	/payments/:paymentId	Get payment receipt by ID	JWT (optional)
GET	/payments/fine/:fineId	Get payment record for a specific fine	JWT
GET	/payments	List all payments (admin)	ADMIN

Simplified Payment Flow

Step 1	Driver calls GET /fines/lookup with referenceNo + categoryCode to retrieve fine details and amount.
Step 2	Driver calls POST /payments/mock-confirm with payment details.
Step 3	Backend validates amount server-side, marks fine as PAID, generates receiptNo, sends SMS to officer.
Step 4	Backend returns SUCCESS + receiptNo in a single response.
Step 5	Driver/app displays the receipt. Officer receives SMS and returns the license.

POST /payments/mock-confirm — Auth: Public / DRIVER

Request Body:

```
{
"fineId": "64f1a2b3c4d5e6f7a8b9c0d4",
"referenceNo": "TF-20260513-000123",
"categoryCode": "SP-01",
"payerName": "Kamal Perera",
"payerPhone": "0771234567",
"payerEmail": "kamal@example.com", // Optional
"paymentMethod": "CARD", // CARD | ONLINE_BANKING | MOBILE_WALLET
"cardNumber": "4111111111111111" // Stored as last 4 digits only
}
```

Response Body:

```
{
"success": true,
"data": {
"paymentId": "64f1a2b3c4d5e6f7a8b9c0d5",
"receiptNo": "RCP-20260513-000456",
"status": "SUCCESS",
"paidAt": "2026-05-13T11:00:00Z",
```

```
"amount": 1500.00,
"fine": {
"referenceNo": "TF-20260513-000123",
"vehicleNo": "CAB-1234",
"category": "Speeding (1-20 km/h over limit)",
"district": "Colombo"
},
"smsSent": true
}
}
```

■ Amount is always taken from the fine record server-side — never trust client-sent amount.
■ Returns 409 if the fine is already PAID. Returns 422 if fine is CANCELLED or OVERDUE policy blocks payment.
■ SMS to officer is triggered synchronously within the same request for simplicity.

5. Notifications /api/v1/notifications

Method	Endpoint	Description	Auth
GET	/notifications/officer/history	SMS send history for the logged-in officer	OFFICER
POST	/notifications/sms/resend/:fineId	Manually resend SMS for a paid fine	OFFICER / ADMIN
GET	/notifications/admin/summary	SMS delivery statistics	ADMIN

SMS is triggered automatically inside POST /payments/mock-confirm — no separate call needed.

SMS Template: Fine TF-20260513-000123 for vehicle CAB-1234 has been PAID (LKR 1,500.00) on 13/05/2026 11:00. Receipt: RCP-20260513-000456. Driver may collect license. — SL Traffic Fine System

6. Analytics /api/v1/analytics

Method	Endpoint	Description	Auth
GET	/analytics/summary	National KPI summary (total collected, pending, etc.)	ADMIN
GET	/analytics/by-district	Collection totals grouped by district	ADMIN
GET	/analytics/by-category	Collection totals grouped by fine category	ADMIN
GET	/analytics/trend	Daily / monthly collection trend	ADMIN
GET	/analytics/top-violations	Top N fine categories by volume	ADMIN
GET	/analytics/officer-stats	Per-officer issue and collection stats	ADMIN
GET	/analytics/district/:id	Drilled-down stats for one district	ADMIN

GET /analytics/summary — Auth: ADMIN

Request Body:

```
Query: ?from=2026-01-01&to=2026-05-31 (defaults: current month)
```

Response Body:

```
{
"success": true,
"data": {
```

```
"totalFinesIssued": 12540,
"totalFinesPaid": 9830,
"totalFinesPending": 2210,
"totalFinesOverdue": 500,
"totalRevenueLKR": 18750000.00,
"pendingRevenueLKR": 4125000.00,
"collectionRate": 78.4,
"periodFrom": "2026-01-01",
"periodTo": "2026-05-31"
}
}
```

GET /analytics/trend — Auth: ADMIN

Request Body:

```
Query: ?from=2026-01-01&to=2026-05-31&granularity=daily|monthly
```

Response Body:

```
{
"success": true,
"data": [
{ "date": "2026-05-01", "finesPaid": 312, "revenueLKR": 567000.00 },
{ "date": "2026-05-02", "finesPaid": 298, "revenueLKR": 541500.00 }
]
}
```

7. Districts /api/v1/districts — Read-Only Reference

> ✂ **Simplified** POST and PUT district endpoints have been REMOVED. Districts are seeded once at startup. Frontend applications hardcode the list. Only GET endpoints are retained for completeness.

Method	Endpoint	Description	Auth
GET	/districts	List all 25 Sri Lankan districts	Public
GET	/districts/:id	Get a single district	Public

GET /districts — Auth: Public

Response Body:

```
{
"success": true,
"data": [
{ "districtId": "64f...", "name": "Colombo", "province": "Western", "code": "CMB" },
{ "districtId": "64f...", "name": "Gampaha", "province": "Western", "code": "GMP" },
{ "districtId": "64f...", "name": "Kandy", "province": "Central", "code": "KDY" }
// ... all 25 districts
]
}
```

■ Frontend apps should cache this list and not call it on every render — it never changes.

8. Users /api/v1/users

Method	Endpoint	Description	Auth
GET	/users	List all users (paginated)	ADMIN
GET	/users/:id	Get user profile by ID	ADMIN
PUT	/users/:id	Update user details	ADMIN / Self
PATCH	/users/:id/status	Activate / deactivate a user account	ADMIN
DELETE	/users/:id	Soft-delete user (sets isActive=false)	ADMIN
GET	/users/officers	List officers (optionally by district)	ADMIN

PATCH /users/:id/status Auth: ADMIN

Request Body:

```
{ "isActive": false }
```

Response Body:

```
{
"success": true,
"data": { "userId": "64f...", "isActive": false }
}
```

9. MongoDB Data Models (Simplified)

✂ Removed refresh_tokens collection removed entirely. No token rotation. JWT_SECRET / JWT_EXPIRES=24h in .env.

User

```
{ _id, fullName, email (unique), phone, passwordHash, role: [DRIVER|OFFICER|ADMIN],
licenseNo (DRIVER only), badgeNo (OFFICER only),
districtId → Districts (OFFICER only), isActive, lastLoginAt, createdAt, updatedAt }
```

Fine

```
{ _id, referenceNo (unique), vehicleNo, driverName, driverLicense,
categoryId → FineCategories, officerId → Users, districtId → Districts,
amount, status: [PENDING|PAID|OVERDUE|CANCELLED],
location: { address (optional string) }, // no lat/lng
notes, issuedAt, dueDate, paidAt, cancelledAt, cancelledBy, createdAt, updatedAt }
```

Payment

```
{ _id, fineId → Fines (unique), receiptNo (unique),
payerName, payerPhone, payerEmail, amount, currency (LKR),
paymentMethod: [CARD|ONLINE_BANKING|MOBILE_WALLET],
cardLastFour, status: [PENDING|SUCCESS|FAILED],
paidAt, createdAt, updatedAt }
```

FineCategory

```
{ _id, code (unique), name, amount, demeritPoints, description, legalRef,
isActive, createdAt, updatedAt }
```

Notification

```
{ _id, fineId → Fines, paymentId → Payments, officerId → Users,
recipientPhone, message, type: [SMS], status: [PENDING|SENT|DELIVERED|FAILED],
```

```
gateway, gatewayRef, retryCount, sentAt, createdAt, updatedAt }
```

District

```
{ _id, name (unique), code (unique), province, isActive }
```

10. Suggested Team Allocation (4 Members)

Member	Responsibility	API Sections
Member 1 (Backend Lead)	Express app setup, JWT auth middleware (single token, 24h), MongoDB connection, deployment	§1 Auth §7 Districts §8 Users
Member 2 (Mobile + Fines)	Fine issuance API, public fine lookup, React Native mobile app (on-the-spot payment)	§2 Fine Categories §3 Fines
Member 3 (Web Portal + Payments)	Mock payment endpoint, SMS trigger, public web portal SPA (React) for online payment	§4 Payments §5 Notifications
Member 4 (Admin Dashboard)	Analytics aggregation pipelines, admin web portal (React), charts (Recharts / Chart.js)	§6 Analytics Admin Portal UI

Mono-Repo Structure: /backend /mobile (React Native / Expo) /web-portal (React SPA) /admin-portal (React SPA) /README.md

Branch Strategy: main (always deployable) → dev → feature/auth · feature/fines · feature/payments · feature/admin